

QUEUES

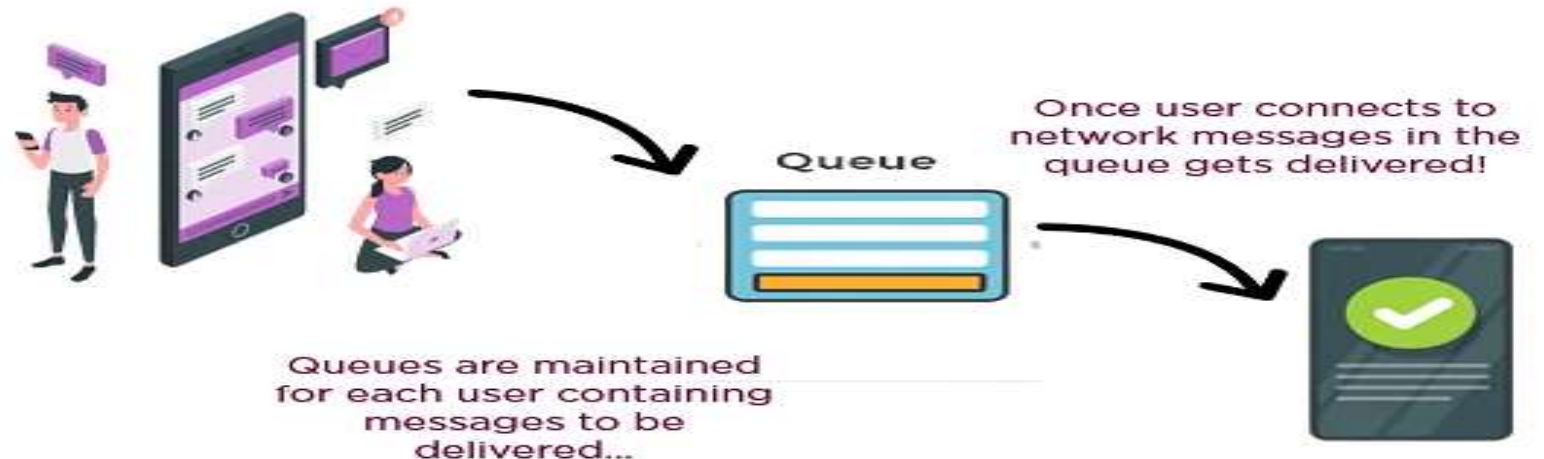
Dr. Jyothi Pillai
Professor
Bhilai Institute of Technology Durg(C.G.)

- Nowadays, several virtual messaging applications serve the obligation of keeping in touch with friends and family.
- These applications maintain the hierarchy of text messages irrespective of whether the person is online or offline.

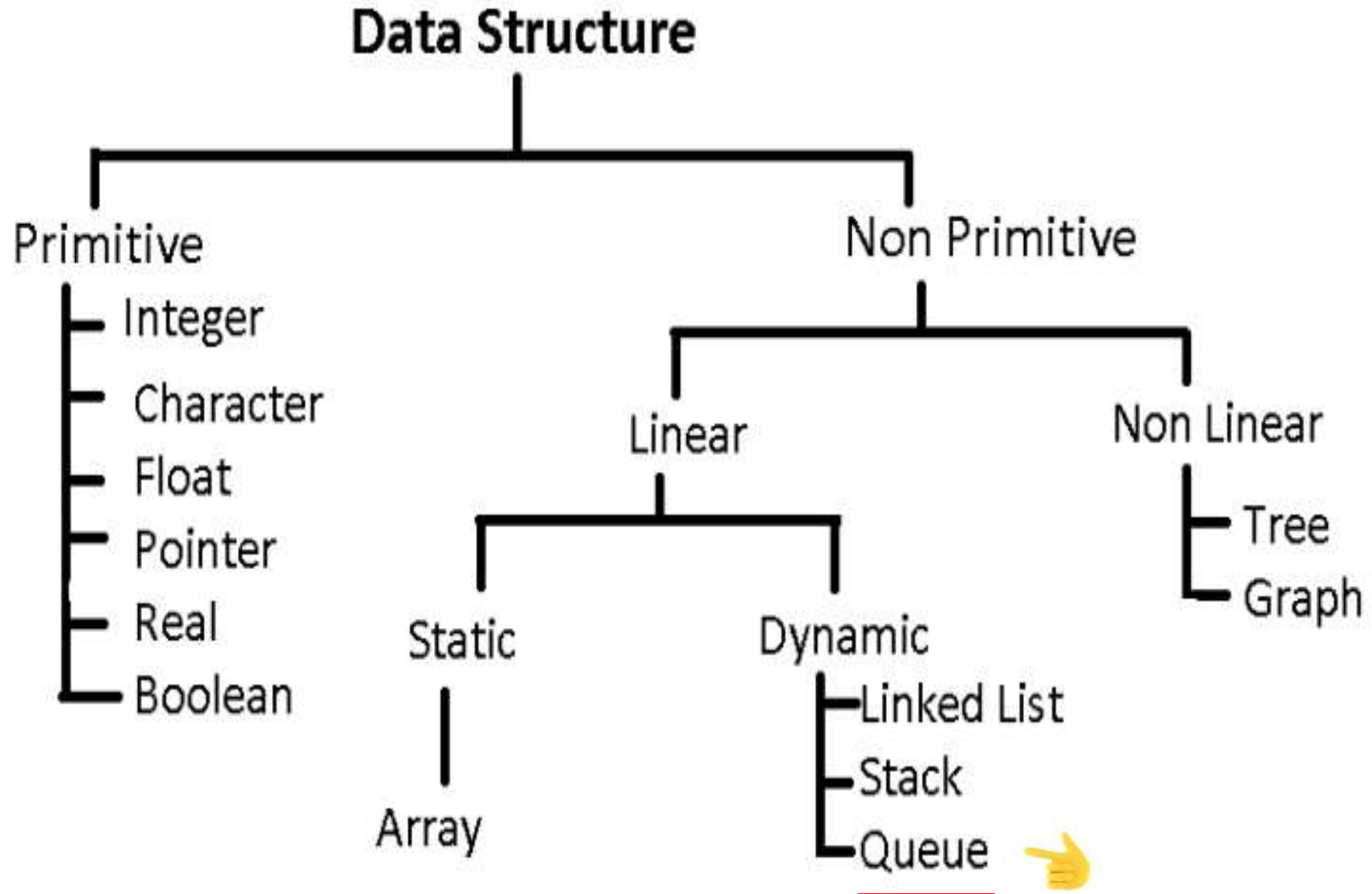


- Question: *how are these messaging applications maintaining the order of text messages?*
- The answer to this question lies in the queue of data structures.
- All messaging applications maintain a queue for each user, containing messages to be delivered to user.
- When user connects to network, messages in queue get delivered. And later, empty queue is deleted.

This example clearly illustrates the importance of queues.



QUEUE



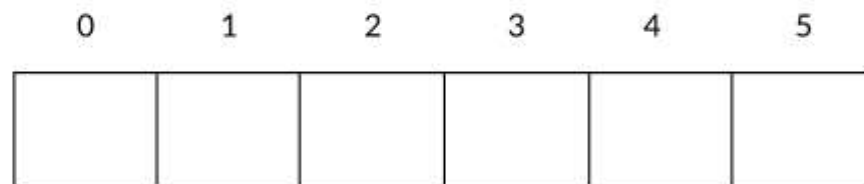
QUEUE

- ❖ A queue is an ordered list which enables insert operations to be performed at one end called **REAR** and delete operations to be performed at another end called **FRONT**.
- ❖ Queue is referred as **First In First Out list(FIFO)** or **Last in Last out (LILO)**.
- ❖ For example, people waiting in line for a rail ticket form a queue.



Queue Operations

Front = Rear = -1

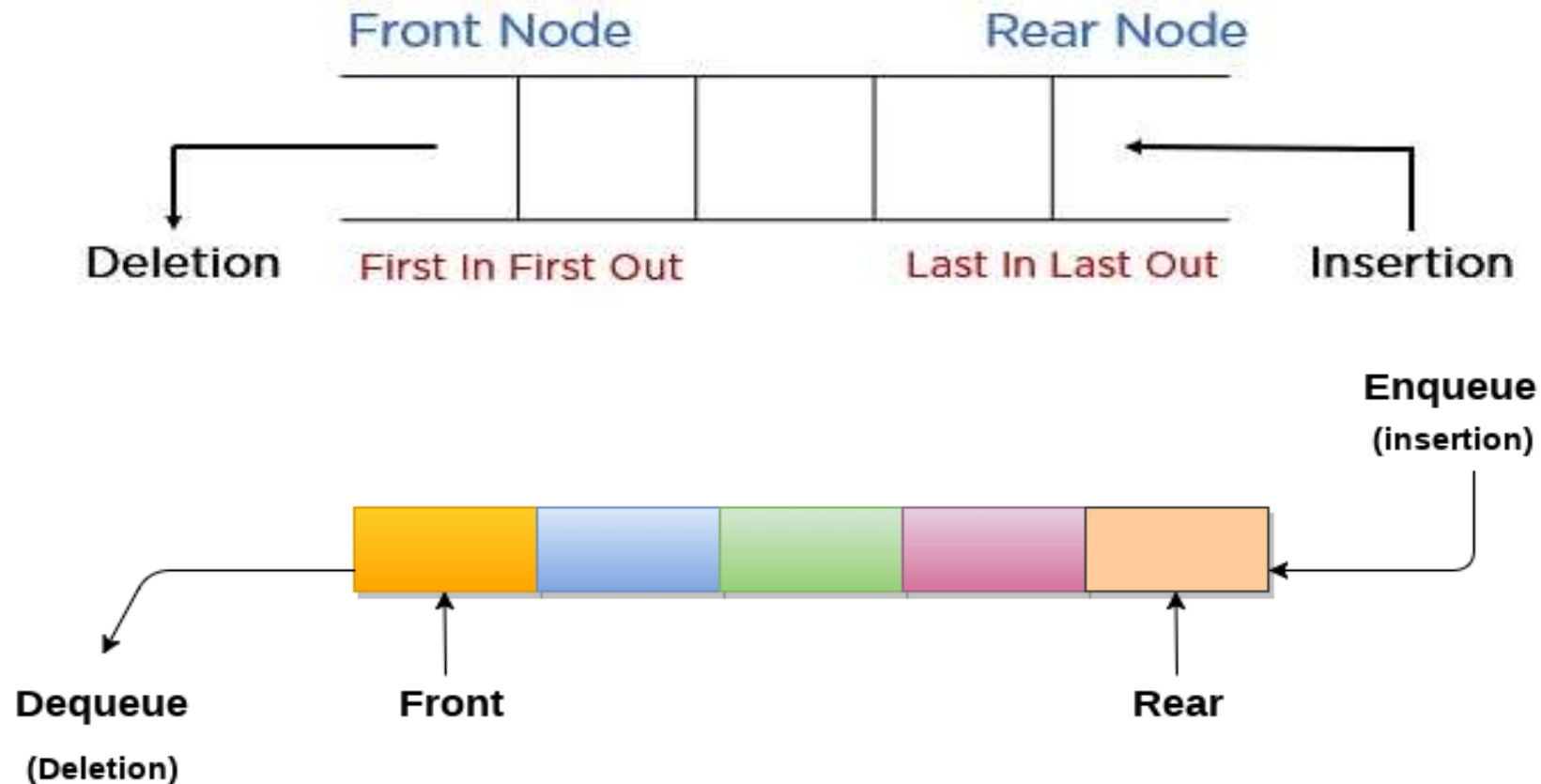


Empty Queue

QUEUE

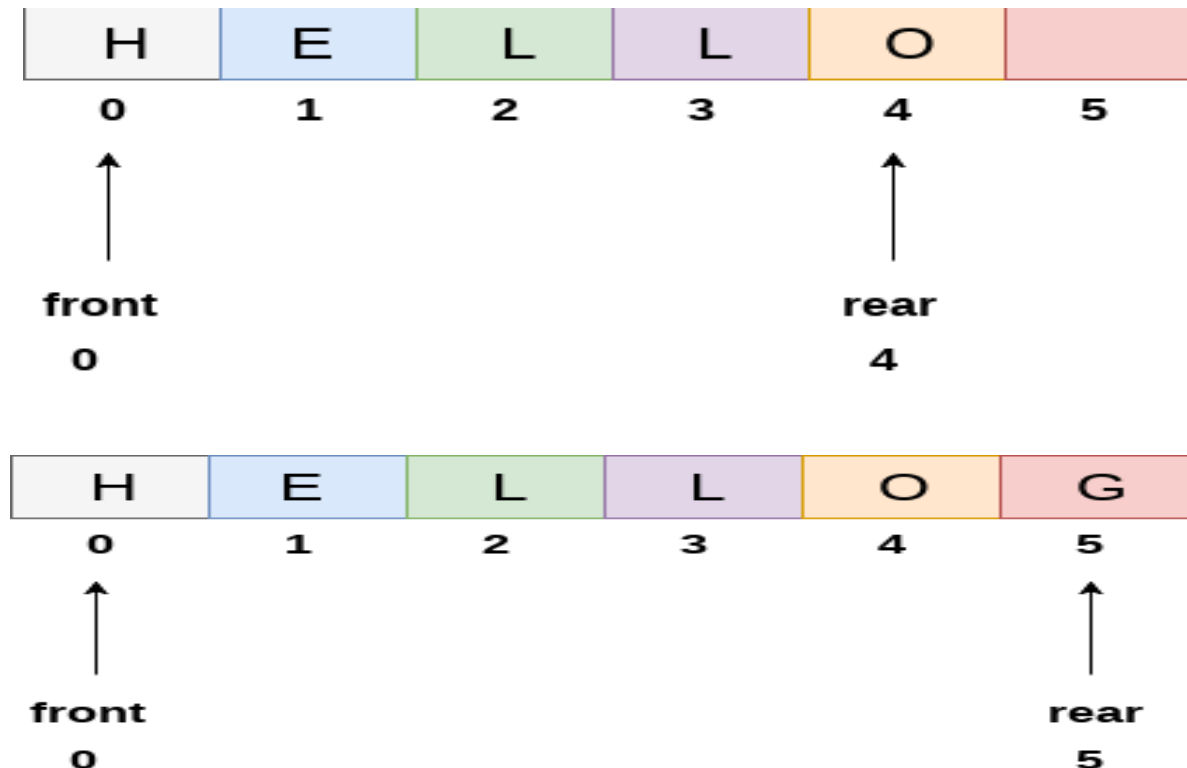
MAIN QUEUE OPERATIONS

- ❖ **EnQueue:** The act of adding a new element to a queue.
- ❖ **DeQueue:** The act of deleting an item from a queue.



MAIN QUEUE OPERATIONS

- ❖ **EnQueue**: When an element is inserted to a queue, the concept is called EnQueue.
- ❖ Enqueue adds an element to the queue, similar to the Push operation of stack.
- ❖ New item enters at Rear/Tail of the queue and is added at Rear end of the queue.
- ❖ EnQueueing an element in a full queue is called **overflow**.



EnQueue

Steps to enqueue (insert) data into a queue –

- ❖ **Step 1**– Check if the queue is full.
- ❖ **Step 2**– If queue is full, produce **overflow error** and exit.
- ❖ **Step 3**– If queue is not full, increment **rear** pointer to point to the next empty space.
- ❖ **Step 4**– Add data element to the queue location, where the rear is pointing.
- ❖ **Step 5**– Return success.

Algorithm

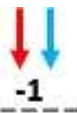
```
procedure enqueue(data)
    if REAR = MAX - 1
        return Overflow
    endif (FRONT = -1 and REAR = -1)
        SET FRONT = REAR = 0
    else
        SET REAR = REAR + 1
    queue[REAR] ← data
        return true
end procedure
```

Program

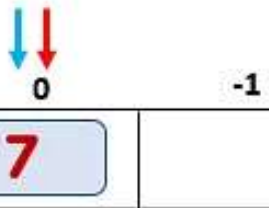
```
int enqueue(int data)
{ if(isfull())
    {return 0;}
elseif(front=-1 && rear=-1)
    {front=rear=0;}
else
    {rear = rear + 1;}
queue[rear] = data;
return 1;
}
```

EnQueue

1. Empty Queue



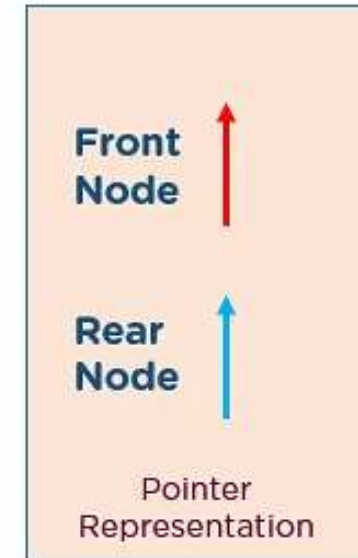
2. Enqueue(7)



3. Enqueue(2)

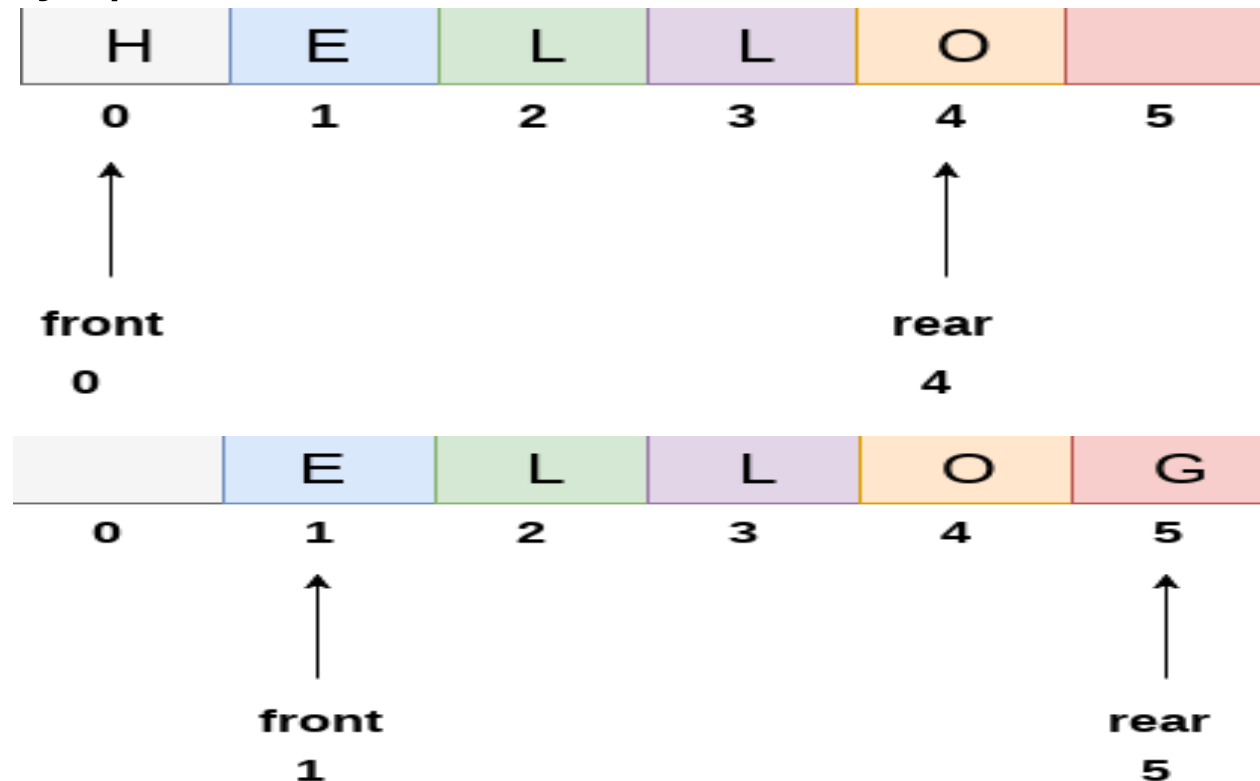


4. Enqueue(-9)



MAIN QUEUE OPERATIONS

- ❖ **DeQueue:** When an element is removed from a queue, the concept is called DeQueue.
- ❖ Dequeue deletes an element to the queue, similar to the Pop operation of stack.
- ❖ Dequeue operation returns and deletes the Front/Head element from the queue.
- ❖ DeQueueing an empty queue is called underflow.



DeQueue

Steps to dequeue (delete) data from a queue –

- ❖ **Step 1** – Check if the queue is empty.
- ❖ **Step 2** – If the queue is empty, produce underflow error and exit.
- ❖ **Step 3** If the queue is not empty, access the data where **front** is pointing.
- ❖ **Step 4** – Increment **front** pointer to point to the next available data element.
- ❖ **Step 5** – Returns success.

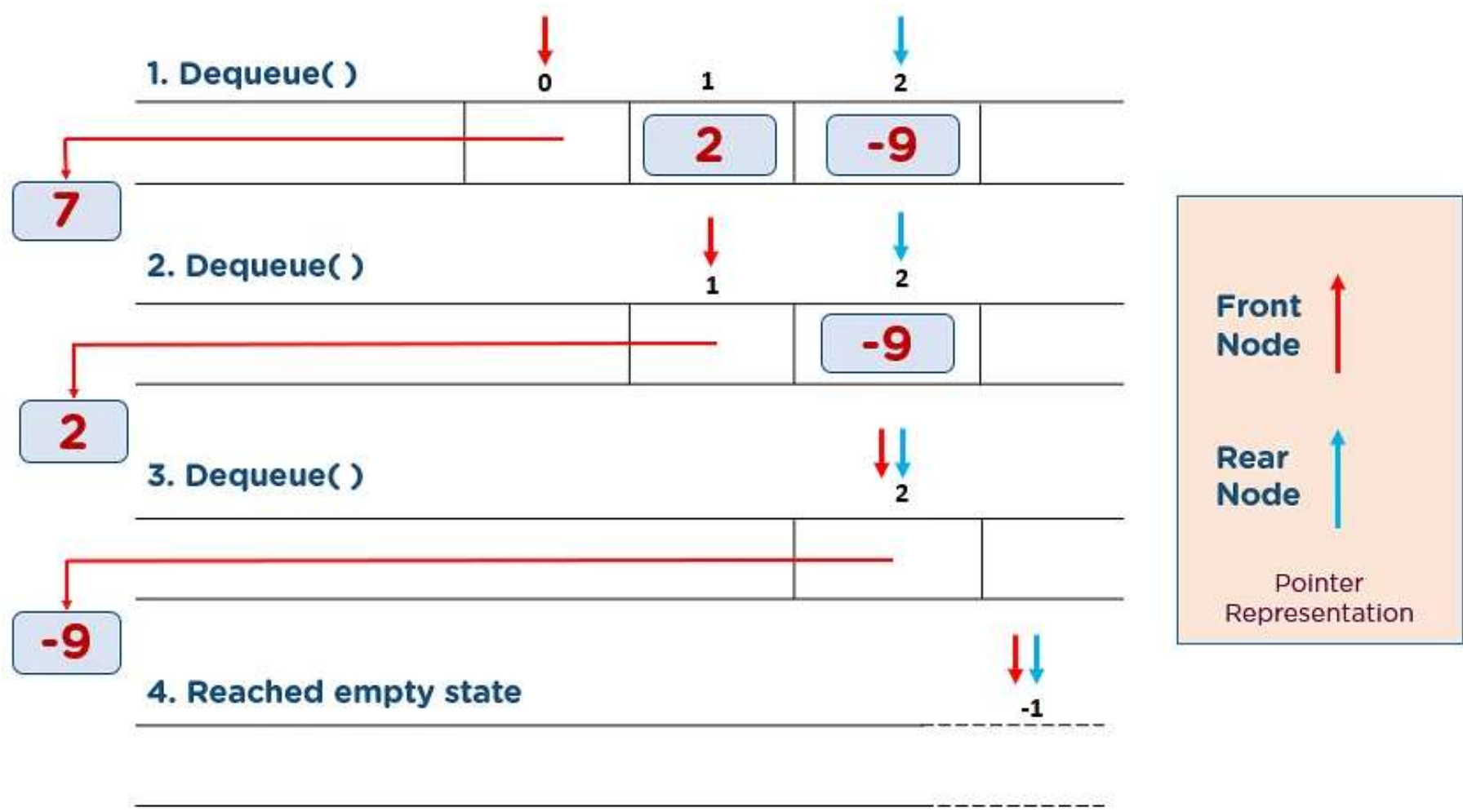
Algorithm

```
procedure dequeue
    if FRONT = -1 or FRONT > REAR
        return underflow
    end if
    data = queue[FRONT]
    FRONT ← FRONT + 1
    return true
end procedure
```

C program

```
int dequeue()
{
    if(isempty())
        return 0;
    int data = queue[front];
    front = front + 1;
    return data;
}
```

DeQueue



C Implementation of EnQueue and DeQueue

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <stdbool.h>
#define MAX 6
int intArray[MAX];
int front = 0;
int rear = -1;
int itemCount = 0;
bool isFull()
{ return itemCount == MAX; }
bool isEmpty()
{ return itemCount == 0; }
int removeData(){
    int data = intArray[front++];
    if(front == MAX)
    { front = 0; }
    itemCount--;
    return data;
}
```

```
int main(){
    int i;

    /* insert 5 items */
    insert(3);
    insert(5);
    insert(9);
    insert(1);
    insert(12);
    insert(15);
    printf("Queue: ");
    for(i = 0; i < MAX; i++)
        printf("%d ", intArray[i]);

    // remove one item
    int num = removeData();
    printf("\nElement removed: %d\n",num);
    printf("Updated Queue: ");
    while(!isEmpty()) {
        int n = removeData();
        printf("%d ",n);
    }
}
```

Output

Queue: 3 5 9 1 12 15

Element removed: 3

Updated Queue: 5 9 1 12 15

PERFORMANCE

- Let “n” be the number of elements in the queue.
- The complexities of queue operations are:-

Representation can be given as:	
Time Complexity of EnQueue()	O(1)
Time Complexity of DeQueue()	O(1)
Time Complexity of QueueSize()	O(1)
Time Complexity of IsEmptyQueue()	O(1)
Time Complexity of IsFullQueue()	O(1)

AUXILIARY QUEUE OPERATIONS

- ❖ **isFull** → It is used to check whether the queue is full or not.
- ❖ **isEmpty** → It is used to check whether the queue has any element or not.
- ❖ **Front** → It is similar to the top operation of a stack i.e., it returns the front element of the queue (but don't delete it)

ISFULL

- To check whether the queue is full or not.

Algorithm

```
begin procedure isfull
    if rear equals to MAXSIZE
        return true
    else
        return false
    endif
end procedure
```

C program

```
bool isfull()
{
    if(rear == MAXSIZE)
        return true;
    else
        return false;
}
```

ISEMPTY

- To check if queue is empty

Algorithm

```
begin procedure isempty
    if front is less than MIN OR
       front is greater than rear
       return true
    else
       return false
    endif
end procedure
```

C program

```
bool isempty()
{
    if(front=-1 && rear=-1)
        return true;
    else
        return false;
}
```


PEEK

- To see the data at the front of the queue, without removing it.

Algorithm

```
begin procedure peek  
    return queue[front]  
end procedure
```

C program

```
int peek()  
{    return queue[front];  
}
```

APPLICATIONS



Printers and single shared resources



Operating Systems for task scheduling



Switches and Routers



Call Center phone systems

- **Printers:** To maintain the order of pages while printing.
- **Asynchronous data transfer** (file IO, pipes, sockets).
- **Interrupt handling** in computers: The interrupts are operated in the same order as they arrive, i.e., interrupt which comes first, will be dealt with first.
- **Process scheduling** in Operating systems: To implement round-robin scheduling algorithms in Multiprogramming.
- **Switches and Routers:** Both switch and router interfaces maintain ingress (inbound) and egress (outbound) queues to store packets.
- **Customer service systems:** Simulation of real-world queues such as call center phone systems. lines at a ticket counter or other first-come first-served scenario requires a queue.

CIRCULAR QUEUE

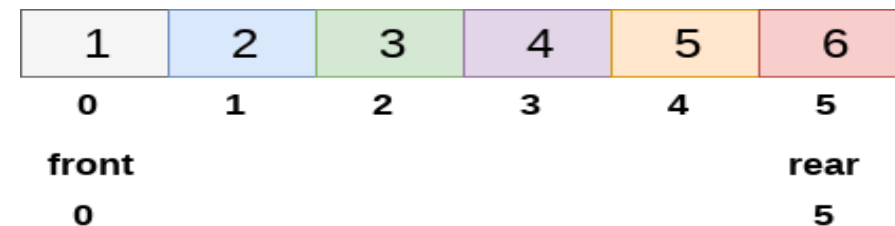
- ❖ In a Linear queue, once the queue is completely full, it's not possible to insert more elements.

Deletions and insertions can only be performed only at front and rear end respectively, as far as linear queue is concerned.

- ❖ Even if we dequeue the queue to remove some of the elements, until the queue is reset, no new elements can be inserted.

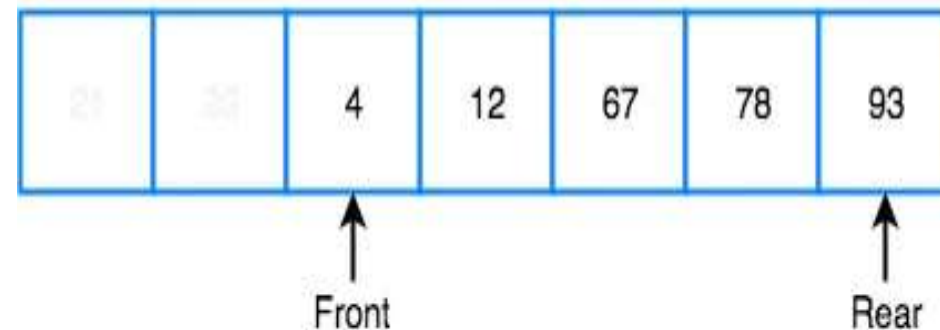
- ❖ When we dequeue any element to remove it from the queue, we are actually moving the front of the queue forward, thereby reducing the overall size of the queue.

- ❖ New elements cannot be inserted because the rear pointer is still at the end of the queue.



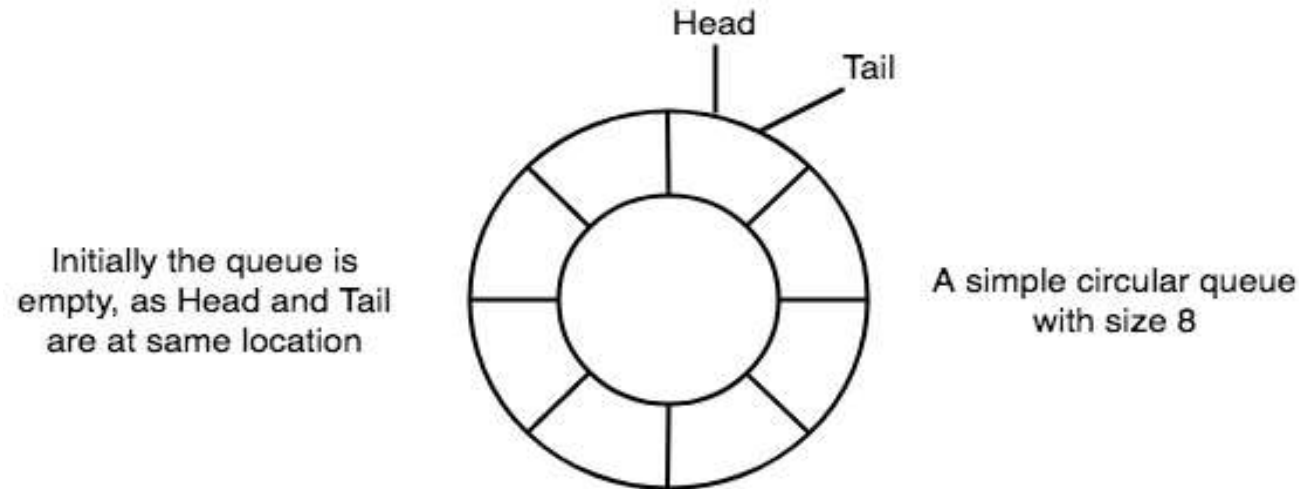
Queue

Queue is Full (Even after removing 2 elements)



CIRCULAR QUEUE

- ❖ Circular queue tries to overcome the problems of Linear Queue.
- ❖ In Circular Queue, memory is utilized when any element is deleted by using that free position.
- ❖ In the circular queue, the first index comes right after the last index.



- ❖ Circular Queue is also a linear data structure, which follows the principle of **FIFO(First In First Out)**, but instead of ending the queue at the last position, it again starts from the first position after the last, hence making the queue behave like a circular data structure.
- ❖ In circular queue, once Queue is full the "**First**" element of the Queue becomes the "**Rear**" element, if and only if the "**Front**" has moved forward; otherwise it will again be a "**Queue Overflow**" state.
- ❖ It is also called as "**Ring buffer**".

LINEAR QUEUE VERSUS CIRCULAR QUEUE

LINEAR QUEUE

A linear data structure that stores data as a sequence of elements similar to a real-world queue

Possible to enter new items from the rear and remove the items from the front

Requires more memory

Less efficient

Queue

CIRCULAR QUEUE

A linear data structure in which the last item connects back to the first item forming a circle

Possible to enter and remove elements from any position

Requires less memory

More efficient

9/11/2023

OPERATIONS ON CIRCULAR QUEUE

1. Front: Get the front item from queue.
2. Rear: Get the last item from queue.
3. enQueue(value) To insert a new element into the circular queue at Rear position -
 - i. Check whether queue is Full – Check((rear == SIZE-1 && front == 0) || (rear==front-1)).
 - ii. If Queue is full then display Queue is Full.
 - iii. If Queue is not full then, if (rear == SIZE – 1 && front != 0) then set rear=0 and insert element.
4. deQueue() To delete an element from the front position of the circular queue-
 - i. Check whether queue is Empty means check (front==-1).
 - ii. If Queue is empty then display Queue is empty else Go to step 3
 - iii. If (front==rear) then set front=rear= -1 elseif (front==size-1) then set front=0 and return the element.

EnQueue

There are three scenarios of inserting an element in a queue.

- 1.If $(\text{rear} + 1) \% \text{maxsize} == \text{front}$ then queue is full. Overflow occurs and insertion can not be performed.
- 2.If $\text{rear} \neq \text{maxsize} - 1$, then **rear** will be incremented to the **mod(maxsize)** and the new value will be inserted at the **rear** end of the queue.
- 3.If $\text{front} \neq 0$ and $\text{rear} == \text{maxsize} - 1$, then queue is not full, set **rear** value to 0 and insert new element.

Algorithm

```
Step 1: If (REAR+1)%MAX==FRONT
        Then Write " Overflow"
        Go to Step 4 [End OF IF]
Step 2: If FRONT=-1 and REAR=-1
        Set FRONT=REAR=0
        Else if REAR=MAX-1 and FRONT!=0
        Set REAR=0
        Else
            Set REAR=(REAR+1)%MAX
        [End of If]
Step 3: Set QUEUE[REAR]=VAL
Step 4: Exit
```

C program

```
void enqueue(int x)
{
    If(front==-1 && rear==-1)
    {
        front=rear=0;
        queue[rear]=x;
    }
    else if((rear+1)%N==front)
    {
        printf("Overflow");
    }
    else
    {
        rear=(rear+1)%N;
        queue[rear]=x;
    }
}
```

DeQueue

To delete an element from the circular queue, check the following three conditions:-

1. If **front = -1**, then there are no elements in the queue, i.e. an underflow condition.
2. If only one element in the queue, i.e. **rear = front** then set **rear = front = -1** and queue is deleted completely.
3. If **front = max -1** then, the value is deleted from the front end and front is set to 0. Otherwise, the front is incremented by 1 and then delete the element at the front end.

Algorithm

```
Step 1: If FRONT = -1
        Then Write " UNDERFLOW "
        Goto Step 4  {END of IF}
Step 2: SET VAL = QUEUE[FRONT]
Step 3: IF FRONT = REAR
        SET FRONT = REAR = -1
    ELSE IF FRONT = MAX -1
        SET FRONT = 0
    ELSE
        SET FRONT = FRONT + 1
[END OF IF]
Step 4: EXIT
```

C program

```
void dequeue()
{
    if(front== -1 && rear== -1)
        {printf("underflow"); }
    else if(front==rear)
        {front=rear=-1}
    else
        { //front++
          front=(front+1)%N;
        }
}
```


DISPLAY

C program

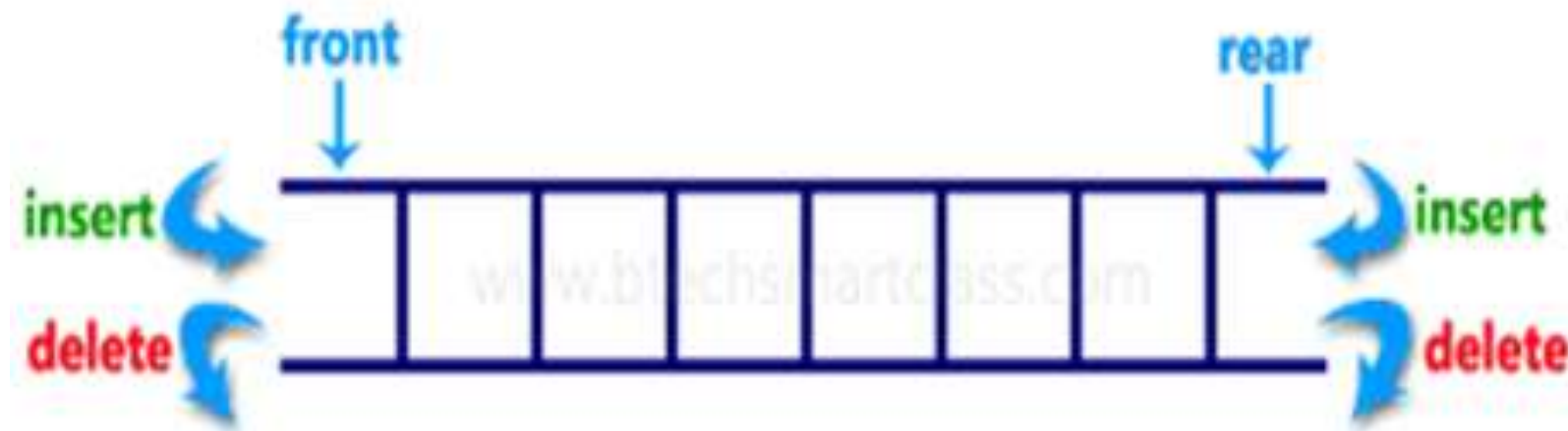
```
void display()
{
    int i=front;

    if(front==-1 && rear==-1)
    {
        printf("Queue is empty");
    }

    else
    {
        while(i!=rear)
        {
            printf("%d",queue[i]);
            i=(i+1)%n;
        }
        printf("%d",queue[rear]);
    }
}
```

DEQUE

- ❖ A **deque**(**Double-ended queue/Deck**) is an ordered collection of items similar to the queue.
- ❖ Deque has two ends; **Front** and **Rear**.
- ❖ What makes a deque different is the unrestrictive nature of adding and removing items.
- ❖ New items can be added at either the front or the rear.
- ❖ Likewise, existing items can be removed from either end.
- ❖ Deque allows elements to be added or removed on both ends.
- ❖ This hybrid data structure provides all capabilities of stacks and queues in single data structure.



TYPES OF DEQUE

❖ Input restricted Deque

- ❖ Elements can be inserted only at one end.
- ❖ Elements can be removed from both the ends.



❖ Output restricted Deque

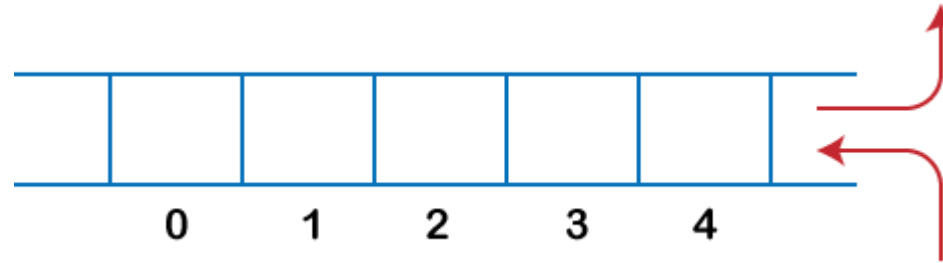
- ❖ Elements can be removed only at one end.
- ❖ Elements can be inserted from both the ends.



DEQUE AS STACK AND QUEUE

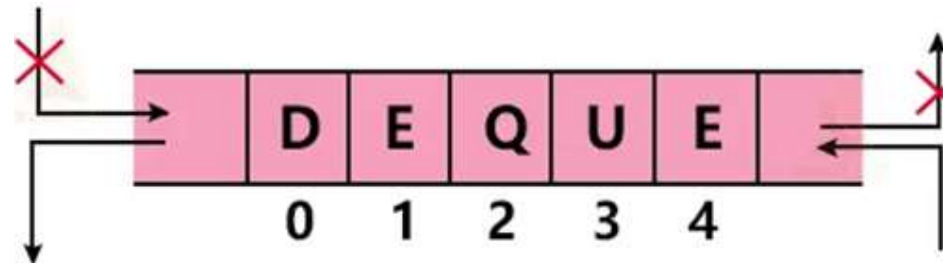
❖ As STACK

- ❖ When insertion and deletion is made at the same side



❖ As Queue

- ❖ When items are inserted at one end and removed at the other end.



DEQUE OPERATIONS

- ❖ `addFront(item)` adds a new item to front of deque; needs item as parameter and returns nothing.
- ❖ `addRear(item)` adds a new item to rear of deque; needs item as parameter and returns nothing.
- ❖ `removeFront()` removes front item from the deque; needs no parameters and returns the item.
- ❖ `removeRear()` removes rear item from the deque; needs no parameters and returns the item.
- ❖ `isEmpty()` tests whether the deque is empty; needs no parameters and returns a boolean value.

DEQUE OPERATIONS

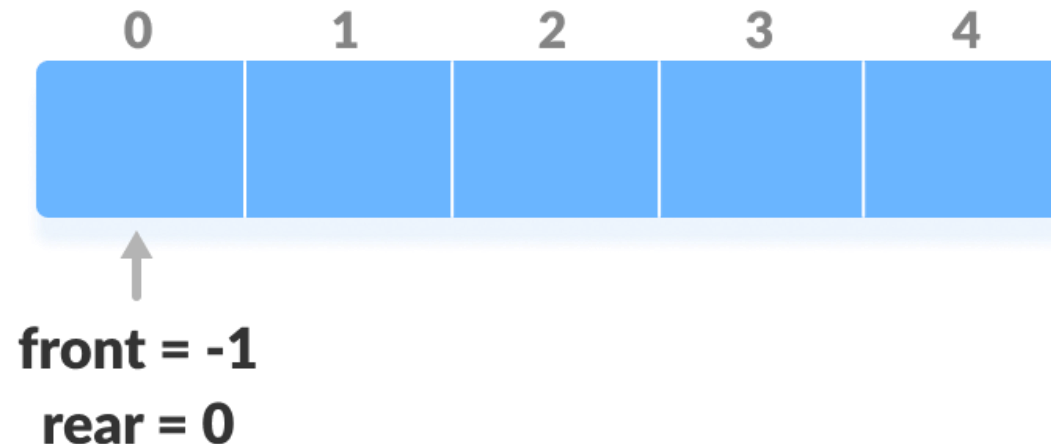
EnQueue

In a linear array implementation of DEQUE, if the array is full, no more elements can be inserted.

In each of the insertion operations, if the array is full, "overflow message" is thrown.

Before performing the insertion operations, these steps are followed.

1. Take an array (deque) of size n .
2. Set two pointers at the first position and set $\text{front} = -1$ and $\text{rear} = 0$.



EnQueueFRONT(INT DATA)

Algorithm

Insert at the Front

Step 1: Check the position of front.



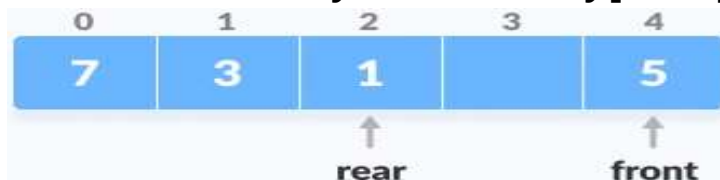
Step 2: If $\text{front} < 1$, reinitialize $\text{front} = n-1$ (last index).



Step 3: Else, decrease front by 1.



Step 4: Add the new key 5 into array[front].



C program

```
Enquefront(int data){  
    if((==N-1) || (front==rear+1))  
        { printf("Q is full");}  
    elseif (front==-1 && rear==-1)  
        { front=0;rear=0;  
          deque[front]=data;  
        }  
    elseif(front==0)  
        { front=N-1;deque[front]=data;  
        }  
    else  
        { front=front-1;deque[front]=data;  
        }  
}
```

EnQueueREAR(INT DATA)

Algorithm

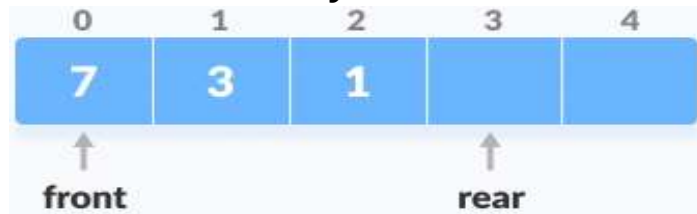
Insert at the Rear

Step 1: Check if the array is full.

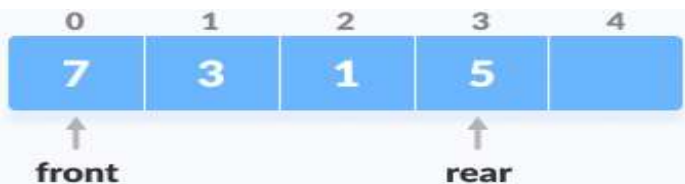


Step 2: If the deque is full, reinitialize rear = 0.

Step 3: Else increase rear by 1.



Step 4: Add the new key 5 into array[rear].



C program

```
Enquerear(int data){  
    if((front==0 && rear==N-1) || (front==rear+1))  
        { printf("Q is full");}  
    elseif (front==-1 && rear==-1)  
        { front=0;rear=0;  
          deque[rear]=data;  
        }  
    elseif(rear==N-1)  
        { Rear=0;deque[rear]=data;  
        }  
    else  
        { Rear=rear+1; dque[rear]=data;  
        }  
}
```


DISPLAY()

C program

```
void display()  
{ int i;i=f;  
  while(i!=rear)  
  {  
    printf("%d",deque[i]);  
    i=(i+1)%n;  
  }  
  printf("%d",deque[rear]);  
}
```

DEQUE OPERATIONS

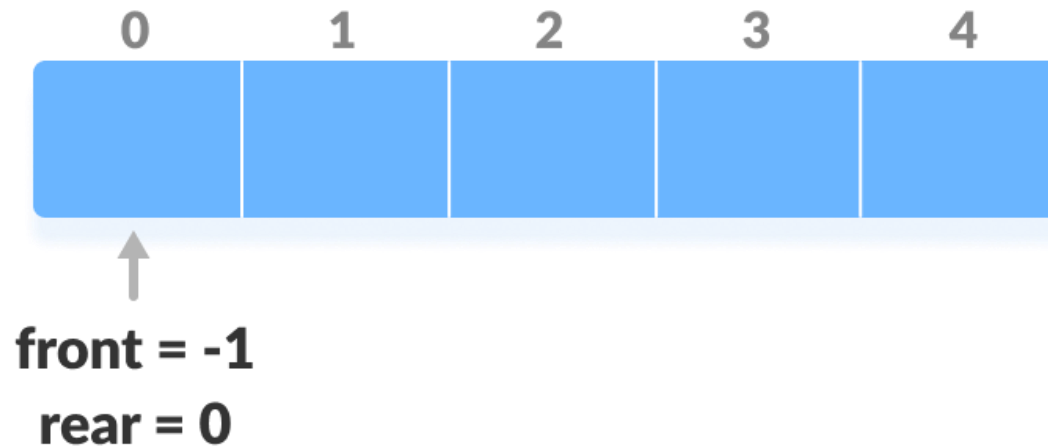
DeQueue

In a linear array implementation of DEQUE, if the array is empty, deletion is not possible.

In each of the deletion operations, if the array is empty, "underrflow message" is displayed.

Before performing the deletion operations, these steps are followed.

1. Take an array (deque) of size n.
2. Set two pointers at the first position and set $\text{front} = -1$ and $\text{rear} = 0$.



DeQueueFRONT()

Algorithm

Delete from the Front

Step 1: Check if the deque is empty.



Step 2: If the deque is empty (i.e. $\text{front} = -1$), deletion cannot be performed (underflow condition).

Step 3: If deque has only one element ($\text{front} = \text{rear}$), set $\text{front} = -1$ and $\text{rear} = -1$.

Step 4: Else if front is at the end (i.e. $\text{front} = n - 1$), set go to the front ($\text{front} = 0$).

Step 5: Else, $\text{front} = \text{front} + 1$.



C program

Dequefront()

```
{ if(front==-1 && rear==-1)
    { printf("Q is MT");
    }
elseif(front==rear)
    { front==-1,rear==-1;}
elseif(front==n-1)
    { front=0;}
else
    { front=front+1;
    }
}
```

DeQueREAR()

Algorithm

Delete from the Rear

Step 1: Check if the deque is empty.

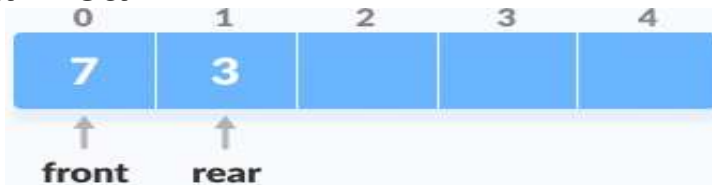


Step 2: If the deque is empty (i.e. front = -1), deletion cannot be performed (underflow condition).

Step 3: If deque has only one element (front = rear), set front = -1 and rear = -1.

Step 4: Else If rear is at the front (i.e. rear = 0), set go to the front (rear = n - 1).

Step 5: Else, rear=rear-1.



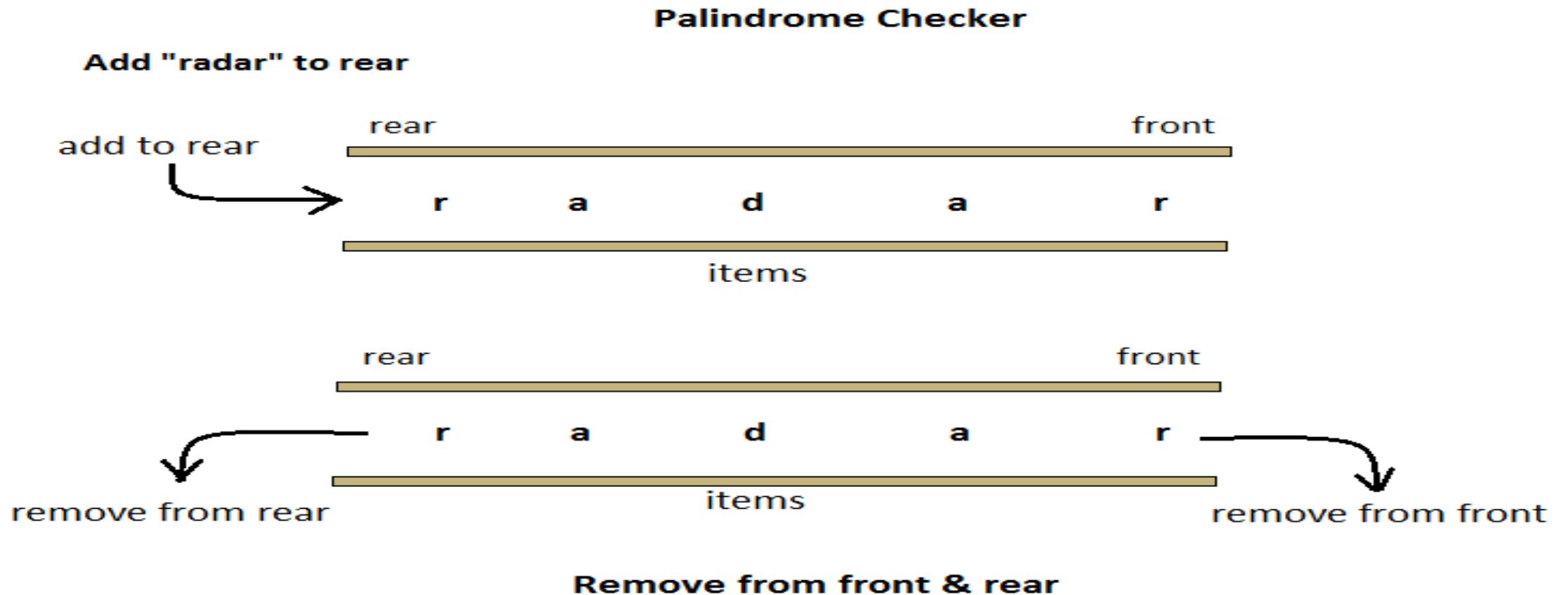
C program

```
DequeRear()
{
    if(front==-1 && rear==-1)
    { printf("Q is MT");
      }
    elseif(front==rear)
    { front==-1,rear==-1;
      }
    elseif(rear==0)
    { rear=n-1;
      }
    else
    { rear=rear-1;
      }
}
```

APPLICATIONS

Palindrome Checker

Madam, Radar, Malayalam

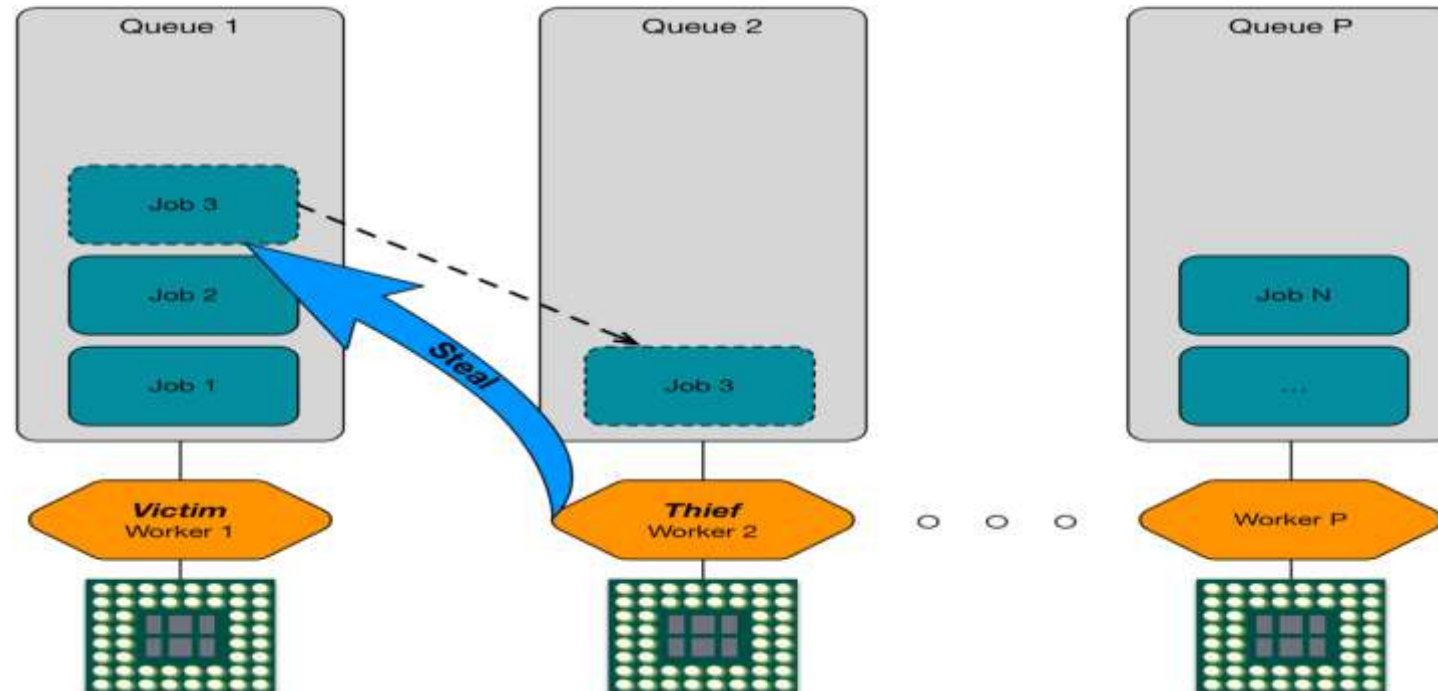


APPLICATIONS

A-Steal job scheduling algorithm

A-Steal algorithm implements task scheduling for several processors(multiprocessor scheduling).

- The processor gets the first element from the deque.
- When one of the processor completes execution of its own threads it can steal a thread from another processor.
- The processor gets the last element from the deque of another processor and executes it.



APPLICATIONS

Undo - Redo operation in software applications

PRIORITY QUEUE

- ❖ Priority queue is an extension of the “normal” queue.
- ❖ It is an abstract data type that contains a group of items.
- ❖ The “normal” queue follows a pattern of first-in-first-out. It dequeues elements in the same order followed at the time of insertion operation.
- ❖ Order of the element in a priority queue depends on element’s priority in that queue.
- ❖ Priority queue moves the highest priority element at the beginning of the priority queue and the lowest priority element at the back of the priority queue.
- ❖ Priority order dequeues those items first that have the highest priority.



CHARACTERISTICS

- ❖ Each item has some priority associated with it.
- ❖ An item with the highest priority is moved at the front and deleted first.
- ❖ If two elements share the same priority value, then the priority queue follows the first-in-first-out principle for dequeue operation.

TYPES OF PRIORITY QUEUE

Ascending Order Priority Queue

- ❖ An ascending order priority queue gives highest priority to the lowest number in the queue.
- ❖ For example, six numbers in the priority queue are 4, 8, 12, 45, 35, 20.
- ❖ Firstly, arrange these numbers in ascending order.
- ❖ The new list is : 4, 8, 12, 20, 35, 45.
- ❖ The ascending order priority queue treats 4 as the highest priority since 4 is the smallest number and 45 as the lowest priority.

4	8	12	20	35	45
---	---	----	----	----	----

TYPES OF PRIORITY QUEUE

Descending Order Priority Queue

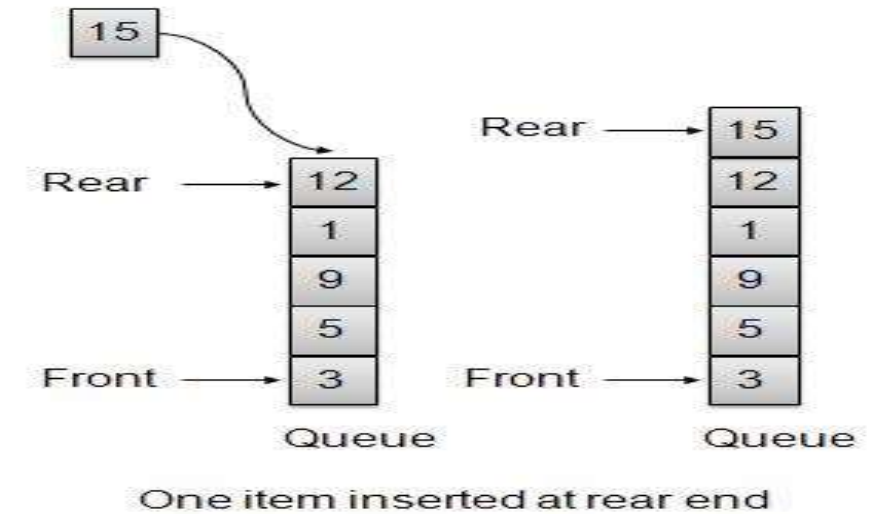
- ❖ A descending order priority queue gives highest priority to the highest number in the queue.
- ❖ For example, six numbers in the priority queue that are 4, 8, 12, 45, 35, 20.
- ❖ Firstly, arrange these numbers in ascending order.
- ❖ The new list is : 45, 35, 20, 12, 8, 4.
- ❖ the descending order priority queue treats 45 as the highest priority since 45 is the highest number and 45 as the lowest priority.

45	35	20	12	8	4
----	----	----	----	---	---

BASIC OPERATIONS

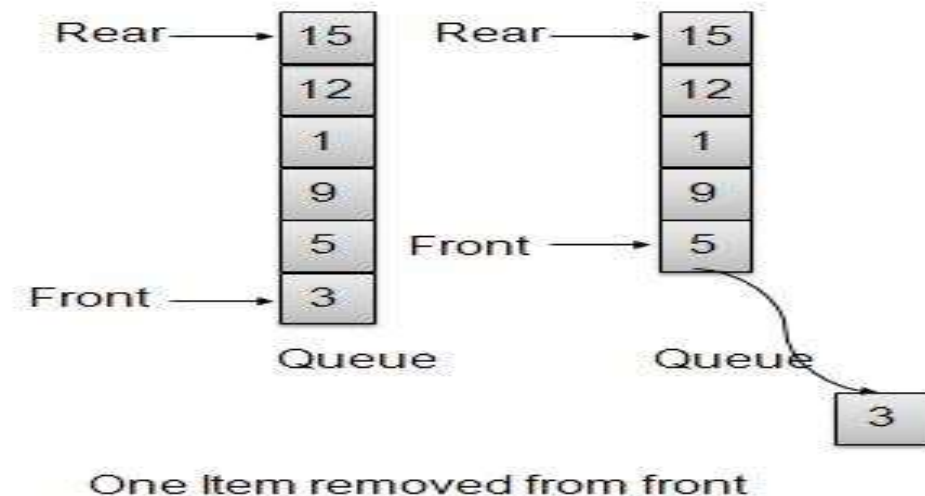
❖ insert / enqueue –

- ❖ add an item to the rear of the queue.
- ❖ When an element is inserted into queue, priority queue inserts the item according to its order.
- ❖ Assuming that data with high value has low priority.



❖ remove / dequeue –

- ❖ remove an item from the front of the queue.
- ❖ When an element is removed from queue, queue get the element using item count.
- ❖ Once element is removed. Item count is reduced by one.



THANK YOU